NEXTCLOUD & EURO-OFFICE

Local (air-gapped) Nextcloud with Euro-Office under Podman

A fully local office and file stack for a government agency — installation record, referenced documents and screenshots

Compiled from the documentation in `content/euro-office` · August 2026

Overview: Local (air-gapped) Nextcloud with Euro-Office under Podman

This document gives an overview of how Nextcloud and Euro-Office were installed on this host as a **fully local** (air-gapped) instance that is reached through the local domain `nextcloud.local`. It is the entry point for the detailed records in this directory:

Document	Content
installation-steps.md	Full install log of Nextcloud All-in-One with Podman
local-nextcloud-fixes.md	The Podman / local-only fixes in detail, plus automation
podman-vs-docker.md	Why Podman is preferred for a government agency
nextcloud-euro-office-integration.md	Euro-Office integration and the document-saving fix
combining-nextcloud-euro-office.md	How Nextcloud AIO bundles Euro-Office as an option
euro-office-github.md	Euro-Office install guide (upstream, GitHub-based)

1. Goal and why a local domain

The aim was a Nextcloud instance that runs **entirely on the local network, with no internet link** - an air-gapped installation. A public domain name can therefore never be used: no Let’s Encrypt certificate can be issued for a LAN-only name, and no external service may be contacted during setup.

To still give the instance a stable, human-friendly address, the local domain **`nextcloud.local`** was mapped to the host’s LAN IP in `/etc/hosts`:

```
192.168.0.114  nextcloud.local
```

Nextcloud All-in-One (AIO) normally insists on a fully qualified public domain and validates it against the public DNS. For a local-only install this is disabled with `SKIP_DOMAIN_VALIDATION=true`, which makes the AIO interface accept any domain without an internet/DNS check. The same principle is described in domain-validation-nextcloud.md.

2. Approach: GitHub repositories as starting point

Instead of running a ready-made installer script, the **source repositories on GitHub** were taken as the starting point and the upstream documentation was followed:

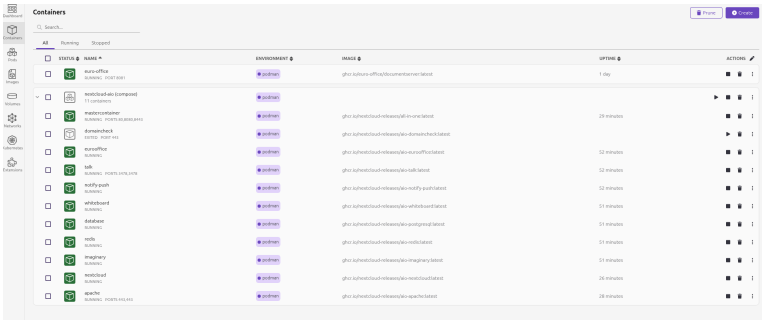
- nextcloud/all-in-one - the AIO mastercontainer that manages the whole Nextcloud stack
- Euro-Office/DocumentServer - the office document server
- Euro-Office/eurooffice-nextcloud - the Nextcloud integration app

Because Podman is not an officially supported engine for AIO, the community guide Rootless Podman Quadlet was used. The `docker run` command from the AIO README was adapted for Podman; the full command is recorded in installation-steps.md.

3. Installation summary

Environment: Linux Mint 22.3, user `naj` (uid/gid 1000), rootless Podman 4.9.3, host LAN IP `192.168.0.114`.

1. **Prerequisites**: rootless Podman with cgroups v2, the Podman API socket (`podman.socket`, exposes the Docker-compatible API that AIO requires), the `podman-restart.service` user unit, and the pulled mastercontainer image.
2. **Mastercontainer**: created with the AIO `docker run` command adapted for Podman (podman socket mount, `DOCKER_API_VERSION=1.41`, `--network bridge`, `SKIP_DOMAIN_VALIDATION=true`).
3. **Browser setup**: the AIO interface is reached at `https://192.168.0.114:8080` (by IP, never by domain, to avoid HSTS issues). The domain `nextcloud.local` is entered without validation, then the required containers (Nextcloud, Database, Redis, Apache) and optional add-ons are started.



Overview: containers in Nextcloud (AIO interface)

4. Fixes required for a functional Nextcloud under Podman

Several things had to be fixed to get the installation to work. They are documented in detail in local-nextcloud-fixes.md and collected in the troubleshooting section of installation-steps.md:

1. **Podman socket permissions** - AIO's `www-data` user must be able to talk to the container engine socket. The socket had been created with mode `0600` instead of the declared `0660`; `systemctl --user restart podman.socket` recreates it correctly.
2. **Docker API version mismatch** - AIO defaults to Docker API `v1.44`, while Podman 4.9.3 exposes `v1.41`. Without `DOCKER_API_VERSION=1.41` the mastercontainer refuses to start.
3. **Non-root containers cannot bind port 443** - Docker grants non-root processes `CAP_NET_BIND_SERVICE`; Podman grants no effective capabilities to non-root users by default, so the `www-data` user in the domaincheck/apache containers got `bind() [::]:443: Permission denied`. Fix: patch `containers.json` inside the mastercontainer to add `CAP_NET_BIND_SERVICE` to those two containers. (Setting `net.ipv4.ip_unprivileged_port_start=80` via `default_sysctls` does not help - the Docker-API path ignores it.)
4. **Self-signed TLS for a local-only domain** - the apache container's Caddy tries to obtain a Let's Encrypt certificate for `nextcloud.local`, which can never succeed, so HTTPS fails with `SSL_ERROR_INTERNAL_ERROR_ALERT`. Fix: bind-mount a custom Caddyfile template that uses Caddy's **internal CA** (`tls { issuer internal }`), which issues a self-signed certificate valid for the domain.

5. **Stale php-fpm `listen.allowed_clients`** - recreating the apache container gives it a new IP in the `nextcloud-aio` bridge network; the Nextcloud container only accepts php-fpm connections from the apache IP recorded at its own start, causing HTTP 503. Fix: `podman restart nextcloud-aio-nextcloud`.

The first four items are applied automatically by the helper script `~/.local/bin/nextcloud-aio-podman-fix.sh`, which is installed as the systemd user service `nextcloud-aio-podman-fix.service` and runs at every session start. It patches `containers.json` idempotently and restarts the mastercontainer only when something changed.

> The `containers.json` patches are **lost whenever the mastercontainer is recreated** (e.g. when updating). Re-run the helper script afterwards.

5. Why Podman is the better choice for a government agency

Podman was chosen over Docker deliberately; podman-vs-docker.md argues the case in full. The short version:

- **Least privilege / compliance** - Podman is **rootless by default** (user namespaces), so containers run as unprivileged users - no root daemon socket to grant root-equivalent access, satisfying FedRAMP, NIST SP 800-53, DISA STIGs and SOC 2 audits.
- **Smaller attack surface** - Podman is **daemonless** (fork/exec model); there is no central `dockerd` whose compromise or crash would affect every container on the host.
- **Zero licensing risk** - Podman is free and open source (Apache 2.0). Docker Desktop requires paid subscriptions for larger organizations, creating audit and procurement overhead.
- **Government Linux alignment** - Podman is the **default container engine in RHEL 8+**, is FIPS-compliant, and integrates natively with `systemd` (this installation uses `systemd` user units and user linger throughout).
- **Auditability** - because containers run as child processes of the invoking user, `auditd` can track individual user IDs down to container events, simplifying forensics.

- **Drop-in compatibility** - Podman is OCI-compliant, so the same images and `Containerfile` / `Dockerfile` workflows keep working (`alias docker=podman`).

6. Euro-Office integration

The last step was integrating **Euro-Office**, an office suite tailored for government agencies. It has stripped out all suspect software and linkages found in other office suites, making it suitable for an air-gapped government environment. See nextcloud-euro-office-integration.md for the full record.

Euro-Office is deployed in two parts (see euro-office-github.md):

1. **Document server** - the real-time collaboration server for office documents.
2. **Nextcloud app** (`eurooffice`) - the integration layer that opens and saves documents inside Nextcloud.

Because AIO ships Euro-Office as a selectable add-on container (combining-nextcloud-euro-office.md), the document server runs as the AIO-managed container `nextcloud-aio-eurooffice` on the `nextcloud-aio` network, reachable in the browser at `https://nextcloud.local/eurooffice`. The app is configured with `DocumentServerUrl = https://nextcloud.local/eurooffice` and a shared JWT secret injected by AIO.

The saving fix

Opening documents worked, but **saving failed** with "The document cannot be saved". The documentserver's background services (converter, docservice) call back into Nextcloud over `https://nextcloud.local`, which is served with a **self-signed certificate** and resolves to a **private IP** - the TLS handshake failed with `UNABLE_TO_GET_ISSUER_CERT_LOCALLY`.

Fix: two settings that map to the documentserver's `local.json`:

Env var	Effect
`USE_UNAUTHORIZED_STORAGE=true`	`requestDefaults.rejectUnauthori = false` (accept the self-signed certificate)

Env var	Effect
`ALLOW_PRIVATE_IP_ADDRESS=true`	`request-filtering-agent.allowPrivateIPAddress = t` (allow the private IP as callback target)

These were applied durably in `containers.json` (survives container recreates) and immediately in the running container's `local.json`. The helper script `nextcloud-aio-podman-fix.sh` was extended to (re-)apply the two env vars after every mastercontainer recreate.

> Disabling `rejectUnauthorized` is acceptable only for this local-only setup with a self-signed certificate. If the instance ever moves to a public domain with a real Let's Encrypt certificate, the variables can be removed again so TLS verification stays enabled.

Result

With the fixes applied, Euro-Office documents can be opened and saved from Nextcloud through the local domain `nextcloud.local` - a fully local, air-gapped office and file stack for a government agency, running rootless under Podman.

Nextcloud All-in-One installation steps (with Podman)

This document records the steps performed to install Nextcloud All-in-One using **Podman** instead of Docker on this host. The Podman and local-only fixes applied here are summarized in local-nextcloud-fixes.md.

Environment

- OS: Linux Mint 22.3 (Ubuntu 24.04 based), kernel 7.0.0
- User: `naj` (uid/gid 1000), member of the `sudo` group
- Podman: `4.9.3` (rootless, cgroups v2, `systemd` cgroup manager, `netavark` network backend)
- Conmon: `2.1.10`, crun: `1.14.1`
- Host LAN IP: `192.168.0.114`

> Podman is not an officially supported container engine for Nextcloud AIO (see the AIO README). This installation follows the community guide Rootless Podman Quadlet and works for the core stack. The built-in Borg backup and automatic mastercontainer self-update are not expected to work under rootless Podman.

0. Prerequisites

1. Podman must be installed and running rootless. Verify with:

   ```
   podman info
   ```

   A rootless setup with cgroups v2 is required (check `podman info | grep -A2 networkBackend` shows `netavark`).

2. Check that no other service uses ports `80`, `8080` or `8443`:

   ```
   ss -tlnp | grep -E ':(80|8080|8443) '
   ```

3. Make sure the Podman API socket is enabled and active (it provides the Docker-compatible API that AIO requires):

```
systemctl --user enable --now podman.socket
systemctl --user is-active podman.socket
```

The socket path is reported by:

```
podman info --format '{{ .Host.RemoteSocket.Path }}'
```

Expected: `/run/user/1000/podman/podman.sock`

4. Enable the user service that restarts containers with `--restart always`:

```
systemctl --user enable podman-restart.service
systemctl --user start podman-restart.service
```

5. Pull the mastercontainer image:

```
podman pull ghcr.io/nextcloud-releases/all-in-one:latest
```

1. Fix the Podman socket permissions

AIO's `www-data` user inside the mastercontainer must be able to talk to the container engine socket. The podman socket unit already declares `SocketMode=0660`, but the running socket had been created with mode `0600`, which broke the check in `Containers/mastercontainer/start.sh`. Restart the socket so it is recreated with the correct group-read/write mode:

```
systemctl --user restart podman.socket
```

Verify:

```
ls -la /run/user/1000/podman/podman.sock
# expected: srw-rw---- 1 naj naj ... podman.sock
```

Optional (persists across reboots): because the socket file is recreated on socket start, you may want to ensure the mode stays `0660` by confirming the socket unit:

```
cat /usr/lib/systemd/user/podman.socket   # contains SocketMode=0660
```

2. Create and start the mastercontainer

The standard `docker run` command from the AIO README, adapted for Podman:

- the Docker socket mount is replaced with the Podman socket
- `WATCHTOWER_DOCKER_SOCKET_PATH` points to the podman socket
- `DOCKER_API_VERSION=1.41` is required because Podman 4.9.3 exposes Docker API `v1.41`, while AIO defaults to `v1.44` and refuses to start otherwise
- `--network bridge` is needed so AIO can create and attach sibling containers to the `nextcloud-aio` network (rootless `slirp4netns` does not support `podman network connect`)
- `SKIP_DOMAIN_VALIDATION=true` is set because this is a **local-only** instance: no internet link is wanted, so the domain is not validated against the public DNS

```
        podman run -d \
          --init \
          --sig-proxy=false \
          --name nextcloud-aio-mastercontainer \
          --restart always \
          --network bridge \
          --publish 80:80 \
          --publish 8080:8080 \
          --publish 8443:8443 \
          --volume nextcloud_aio_mastercontainer:/mnt/docker-
aio-config \
          --volume
/run/user/1000/podman/podman.sock:/var/run/docker.sock:ro \
          --env
WATCHTOWER_DOCKER_SOCKET_PATH=/var/run/docker.sock \
          --env DOCKER_API_VERSION=1.41 \
          --env SKIP_DOMAIN_VALIDATION=true \
          ghcr.io/nextcloud-releases/all-in-one:latest
```

> Do not change the container name `nextcloud-aio-mastercontainer` or the volume name `nextcloud_aio_mastercontainer` - both are required for AIO to work.

3. Verify the installation

```
        podman ps -a | grep nextcloud-aio-mastercontainer
        # STATUS should be "Up (healthy)"

        podman logs nextcloud-aio-mastercontainer | tail -10
        # should end with: "fpm is running" / "ready to handle
connections"
```

Check the HTTP endpoints:

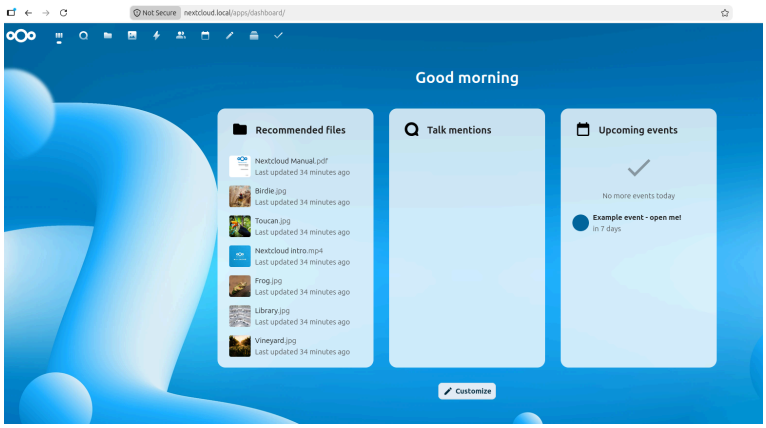
```
        curl -sk -o /dev/null -w "interface: %{http_code}\n"
https://localhost:8080
        curl -s -o /dev/null -w "port 80: %{http_code}\n"
http://localhost:80
```

4. Complete the setup in the browser

1. Open the AIO interface using the **IP address** (never a domain, to avoid HSTS issues):

   ```
   https://192.168.0.114:8080
   ```

2. Accept the self-signed certificate warning.
3. In the AIO interface, when asked for your domain: since `SKIP_DOMAIN_VALIDATION=true` is set, the interface shows **“domain validation is disabled so any domain will be accepted here”** and no internet/DNS check is performed. Use a local-only name (e.g. `nextcloud.local` or the machine’s hostname) — it does not need to resolve publicly.
4. Enter your email address and create the AIO interface password.
5. Select the containers you want (Nextcloud, Database, Redis and Apache are required; optionally Collabora, Talk, etc.) and click **Start containers**.
6. Wait until the AIO interface reports the containers as running, then click the button to open your Nextcloud instance and set up your admin account.



Nextcloud local admin login

> Local-only access caveats: - No valid certificate can be issued for a LAN-only domain, so the Nextcloud page uses a self-signed certificate (issued by Caddy's internal CA, see section 4b) — accept the browser warning, or access it via a reverse proxy that terminates TLS locally (see the reverse proxy docs). - To resolve your chosen domain on LAN clients, add it to the local DNS server or to the `/etc/hosts` of the clients (or use the AIO dnsmasq / pi-hole community containers).

If you enable Nextcloud Talk, open ports `3478/TCP` and `3478/UDP` in the firewall/router.

> Before entering your domain, apply the port-443 fix from section 4a below, otherwise the **domaincheck** container (and later the **apache** container) will fail to start.

4a. Required fix: allow non-root containers to bind port 443

The AIO domaincheck and apache containers run as the non-root user `www-data` and bind host port `443`. Docker grants non-root processes `CAP_NET_BIND_SERVICE`, but **Podman grants no effective capabilities to non-root users by default**. The result is:

```
nextcloud-aio-domaincheck: (lighttpd) bind() [::]:443:
Permission denied
```

(Note: setting `net.ipv4.ip_unprivileged_port_start=80` via `default_sysctls` in `containers.conf` does **not** work here - the Podman Docker-API compatibility path ignores `default_sysctls` for containers created by the mastercontainer.)

Fix: make the mastercontainer add `CAP_NET_BIND_SERVICE` to the domaincheck and apache containers. The container definitions in `containers.json` support a `cap_add` field which the mastercontainer forwards into the container-create request. Since `start.sh` refuses to start if `containers.json` is bind-mounted, patch the file inside the running mastercontainer instead (it lives in the container's writable layer and survives `podman restart`).

A helper script handles this automatically (idempotent; restarts the mastercontainer only when a change was made):

```
        ~/.local/bin/nextcloud-aio-podman-fix.sh
```

The script is installed as an enabled systemd user service so it also runs at every session start:

```
        ~/.config/systemd/user/nextcloud-aio-podman-fix.service
        systemctl --user status nextcloud-aio-podman-fix.service
```

What the script does (equivalent manual steps):

```
        # 1. add "cap_add": ["NET_BIND_SERVICE"] to the entries
of
        #    nextcloud-aio-domaincheck and nextcloud-aio-apache,
and (since the
        #    local-only TLS fix of section 4b) a bind mount of
        #    /home/naj/aio-config/Caddyfile:/Caddyfile:ro to
nextcloud-aio-apache,
        #    all inside containers.json in the mastercontainer
        podman exec nextcloud-aio-mastercontainer python3 -
<<'PYEOF'
        import json
        path = '/var/www/docker-aio/php/containers.json'
        data = json.load(open(path))
        for c in data['aio_services_v1']:
            if c.get('container_name') in ('nextcloud-aio-
domaincheck', 'nextcloud-aio-apache'):
                c.setdefault('cap_add', []).insert(0,
'NET_BIND_SERVICE')
            if c.get('container_name') == 'nextcloud-aio-
apache':
                c.setdefault('volumes', []).append({
                    'source': '/home/naj/aio-config/Caddyfile',
                    'destination': '/Caddyfile',
                    'writeable': False,
                })
        json.dump(data, open(path, 'w'), indent=2)
        PYEOF

        # 2. restart the mastercontainer so the containers.json
cache is cleared
        podman restart nextcloud-aio-mastercontainer
```

> The patch is lost when the mastercontainer container is **recreated** (`podman rm -f` + `podman run` , e.g. when updating per section 5). Re-run `~/.local/bin/nextcloud-aio-podman-fix.sh` afterwards.

To re-trigger the domaincheck after the fix (it is started on every load of the `/containers` page while no domain is set):

```
        podman rm -f nextcloud-aio-domaincheck
        # then load https://192.168.0.114:8080/containers (or
re-submit the domain in the interface)
```

4b. Required fix for local-only setups: self-signed TLS (Caddy internal CA)

With a LAN-only domain like `nextcloud.local` the apache container's TLS endpoint (Caddy) tries to obtain a **Let's Encrypt** certificate via ACME. A `.local` name can never get a public certificate, so Caddy never issues one and fails the TLS handshake instead of falling back:

```
{"level":"error","logger":"tls.obtain","msg":"will retry",
 "error":"[nextcloud.local] Obtain: subject 'nextcloud.local'
does not qualify for a public certificate", ...}
```

In the browser this shows as **`SSL_ERROR_INTERNAL_ERROR_ALERT`** (or `ERR_SSL_PROTOCOL_ERROR`).

Fix: make the apache container use Caddy's **internal CA** (`tls internal`), which issues a self-signed certificate valid for the configured domain. The Caddyfile template lives in the image at `/Caddyfile` and is rendered by `Containers/apache/start.sh`, so the fix is to bind-mount a custom template over it.

1. Create a custom Caddyfile template (a copy of the image's `/Caddyfile`) with the ACME issuer block replaced by the internal issuer:

   ```
           podman exec nextcloud-aio-apache cat /Caddyfile \
             | sed 's|issuer acme {|issuer internal {|;
   s|profile tlsserver||; s|disable_http_challenge||' \
             > /home/naj/aio-config/Caddyfile
           # keep the generated file stable - it is a template
   that start.sh post-processes
           # (lines "auto_https ...", "# trusted_proxies
   placeholder" and
           # "https://{$ADDITIONAL_TRUSTED_DOMAIN}:443," must
   stay intact)
   ```

   The helper script already ships a correct copy at `/home/naj/aio-config/Caddyfile` (the `tls` block reads `tls { issuer internal }`).

2. The fix script `~/.local/bin/nextcloud-aio-podman-fix.sh` additionally adds a read-only bind mount `/home/naj/aio-config/Caddyfile:/Caddyfile:ro` to the apache container definition in `containers.json` (same place as the `cap_add` patch). Run it:

   ```
           ~/.local/bin/nextcloud-aio-podman-fix.sh
   ```

3. Recreate the apache container so it is started with the new bind mount. In the AIO interface click **Stop containers** then **Start containers**. (Note:

password login to the AIO interface is **blocked while the apache container runs** - that is how this was applied below. Alternatively remove apache first to unlock the login: `podman rm -f nextcloud-aio-apache`.)

4. Afterwards the certificate is issued by `Caddy Local Authority - ECC Intermediate` and the handshake succeeds; the browser shows a normal self-signed-certificate warning which you accept. Verify:

```
        echo | openssl s_client -connect nextcloud.local:443
-servername nextcloud.local 2>/dev/null | grep -i 'verify
return code'
```

After the apache container is recreated its IP in the `nextcloud-aio` bridge network **changes**. The Nextcloud container only allows php-fpm connections from the apache IP recorded at its own start (`listen.allowed_clients`), so you get HTTP `503` and log lines like `Connection disallowed: IP address '10.89.3.23' has been dropped.` Fix by restarting Nextcloud so it recomputes the allowed list with the current apache IP:

```
        podman restart nextcloud-aio-nextcloud
```

This happens on every apache recreation (e.g. after a mastercontainer update that recreates all containers together; then a single restart of Nextcloud suffices).

If you later switch to a public domain, remove the bind mount (and the `tls internal` template) again so real Let’s Encrypt certificates can be issued.

5. Management

- Start/stop:

```
        podman start nextcloud-aio-mastercontainer
        podman stop nextcloud-aio-mastercontainer
```

- View logs:

```
        podman logs -f nextcloud-aio-mastercontainer
```

- Update the mastercontainer manually (self-update via Watchtower is not expected to work with rootless Podman):

```
        podman rm -f nextcloud-aio-mastercontainer
        podman pull ghcr.io/nextcloud-releases/all-in-
one:latest
        # then re-run the podman run command from section 2
```

- Restart all AIO containers:

```
        podman ps -a -f 'name=^nextcloud-aio' --
format='{{.Names}}' | xargs podman restart
```

6. Persistence across reboots

- The mastercontainer has `--restart always`, which Podman honors via `podman-restart.service` while the user session runs.
- To have containers start even without an active login session, enable user linger (requires sudo, run interactively):

```
        sudo loginctl enable-linger naj
```

Troubleshooting

- **“Docker socket is not readable by the www-data user”**: the podman socket was created with mode `0600`. Fix with `systemctl --user restart podman.socket` (unit sets `SocketMode=0660`) and restart the mastercontainer.
- **“Docker API v1.44 is not supported by your docker engine”**: Podman exposes a lower Docker API version. Set `--env DOCKER_API_VERSION=1.41` and recreate the mastercontainer. Check the supported version with:

```
        curl -s --unix-socket
/run/user/1000/podman/podman.sock -D - -o /dev/null
http://localhost/_ping | grep -i '^Api-Version'
```

- **Network-related failures when starting sibling containers**: ensure the mastercontainer runs with `--network bridge` so AIO can attach containers to its `nextcloud-aio` network.
- **“Domaincheck container is not running”** / **`bind() [::]:443: Permission denied`**: see section 4a. Podman does not give non-root container users

effective capabilities, so `www-data` cannot bind port 443. Fix by adding `CAP_NET_BIND_SERVICE` to the domaincheck and apache container definitions (`~/.local/bin/nextcloud-aio-podman-fix.sh`). Re-run the fix after recreating the mastercontainer.

- **`SSL_ERROR_INTERNAL_ERROR_ALERT` / `ERR_SSL_PROTOCOL_ERROR` on `https://nextcloud.local`**: see section 4b. Caddy tries ACME for the local-only domain and cannot issue a certificate. Switch the apache container's Caddy to `tls internal` by bind-mounting `/home/naj/aio-config/Caddyfile:/Caddyfile:ro` (done by the fix script), then recreate the apache container.
- **HTTP 503 with `Connection disallowed: IP address '10.89.x.y' has been dropped.` in the Nextcloud logs**: the apache container was recreated and got a new bridge-network IP that is not in php-fpm's `listen.allowed_clients`. Run `podman restart nextcloud-aio-nextcloud`.

Local Nextcloud AIO fixes (Podman)

This document collects the fixes needed to run Nextcloud All-in-One with **Podman** as a **local-only** instance (no public domain, no internet link). It is a companion to installation-steps.md, which contains the full install log.

Two problems were solved:

1. Non-root AIO containers cannot bind host port `443` under Podman.
2. Caddy in the apache container tries to obtain a Let's Encrypt certificate for a LAN-only domain (e.g. `nextcloud.local`), which can never succeed, so HTTPS fails with `SSL_ERROR_INTERNAL_ERROR_ALERT`.

Environment

- Linux Mint 22.3, user `naj` (uid/gid 1000), rootless Podman 4.9.3
- Docker-compatible API via `podman.socket` (exposes API `v1.41`)
- AIO mastercontainer runs with `--network bridge`, `DOCKER_API_VERSION=1.41` and `SKIP_DOMAIN_VALIDATION=true`
- Chosen local domain: `nextcloud.local` (mapped in `/etc/hosts` to `192.168.0.114`)
- AIO interface password: `scolding alfalfa perm greyhound ranger decal juice much`

Fix 1: Allow non-root containers to bind port 443

Problem

The AIO `nextcloud-aio-domaincheck` and `nextcloud-aio-apache` containers run as the non-root user `www-data` and bind host port `443`. Docker grants non-root processes `CAP_NET_BIND_SERVICE`; **Podman grants no effective capabilities to non-root users by default**, so binding fails:

```
nextcloud-aio-domaincheck: (lighttpd) bind() [::]:443:
Permission denied
```

(Setting `net.ipv4.ip_unprivileged_port_start=80` via `default_sysctls` in `~/.config/containers/containers.conf` does **not** help: the Podman Docker-API compatibility path ignores `default_sysctls` for containers created by the mastercontainer.)

Fix

Make the mastercontainer add `CAP_NET_BIND_SERVICE` to the domaincheck and apache container definitions. The definitions live in `containers.json` inside the running mastercontainer (`/var/www/docker-aio/php/containers.json`, in the container's writable layer) and support a `cap_add` field that the mastercontainer forwards into the container-create request.

The patch survives `podman restart` but is **lost when the mastercontainer is recreated** — re-apply afterwards (see the helper script below).

Fix 2: Self-signed TLS for a local-only domain

Problem

The apache container runs Caddy for TLS termination. Its Caddyfile (`/Caddyfile` in the image, rendered to `/tmp/Caddyfile` by `Containers/apache/start.sh`) uses an ACME issuer:

```
tls {
    issuer acme {
        profile tlsserver
        disable_http_challenge
    }
}
```

Caddy tries to obtain a Let's Encrypt certificate for `nextcloud.local`. A `.local` name can never get a public certificate, so Caddy keeps retrying and **never serves TLS**, which shows as `SSL_ERROR_INTERNAL_ERROR_ALERT` / `ERR_SSL_PROTOCOL_ERROR` in the browser. Apache container logs show:

```
{"level":"error","logger":"tls.obtain","msg":"will retry",
 "error":"[nextcloud.local] Obtain: subject 'nextcloud.local'
does not qualify for a public certificate","attempt":1,...}
```

Fix

Replace the ACME issuer with Caddy's internal CA, which issues a self-signed certificate valid for the configured domain. Since the Caddyfile is regenerated on every container start, a custom template is **bind-mounted** into the container at `/Caddyfile`.

Fix 3 (follow-up): stale php-fpm `listen.allowed_clients`

Recreating the apache container gives it a **new IP** in the `nextcloud-aio` bridge network. The Nextcloud container's php-fpm only accepts connections from the apache IP recorded when Nextcloud started (`listen.allowed_clients`), so requests are dropped:

```
ERROR: Connection disallowed: IP address '10.89.3.23' has been dropped.
```

This surfaces as HTTP `503` (Apache `AH01074: Failed writing Environment`). Fix by restarting Nextcloud so its `start.sh` recomputes the allowed list with the current apache IP.

Files

Path	Purpose
`~/.local/bin/nextcloud-aio-podman-fix.sh`	Helper script: idempotently patches `containers.json` (cap_add + Caddyfile bind mount) and restarts the mastercontainer if something changed.
`~/.config/systemd/user/nextcloud-aio-podman-fix.service`	Runs the helper script at session start (after `podman.socket` / `podman-restart.service`).
`/home/naj/aio-config/Caddyfile`	Custom apache Caddyfile template with `tls { issuer internal }`, bind-mounted into the apache container at `/Caddyfile:ro`.
`containers.json` (inside mastercontainer)	Patched container definitions (source of truth for the mastercontainer's container creation).

Applying the fixes

Prerequisites

- The host file `/home/naj/aio-config/Caddyfile` must exist (the script aborts otherwise). Content: copy of the apache image's `/Caddyfile` template, with the ACME issuer block replaced by `tls { issuer internal }`:

```
tls {
    issuer internal
}
```

- The mastercontainer must be running (the script waits up to 60 s for it).

Run the helper script

```
~/.local/bin/nextcloud-aio-podman-fix.sh
```

It:

1. Adds `"cap_add": ["NET_BIND_SERVICE"]` to `nextcloud-aio-domaincheck` and `nextcloud-aio-apache` in `containers.json`.
2. Adds the bind mount `/home/naj/aio-config/Caddyfile:/Caddyfile:ro` to `nextcloud-aio-apache` (if missing).
3. Restarts the mastercontainer only when something changed (clears the `containers.json` cache).

Recreate the apache container

The apache container must be (re)created for the changed definition to take effect:

1. In the AIO interface click **Stop containers**, then **Start containers**.

   Or, because password login to the AIO interface is **blocked while apache runs**:

```
podman rm -f nextcloud-aio-apache      # temporarily unblocks interface login
# then log in at https://192.168.0.114:8080 and click "Start containers"
```

Restart Nextcloud (only if it returns 503)

```
podman restart nextcloud-aio-nextcloud
```

Verify

```
        # 1. bind mount present on the apache container
        podman inspect nextcloud-aio-apache --format '{{json
.HostConfig.Binds}}'
        # -> contains "/home/naj/aio-
config/Caddyfile:/Caddyfile:ro"

        # 2. rendered Caddyfile uses the internal issuer
        podman exec nextcloud-aio-apache cat /tmp/Caddyfile |
grep -A2 'TLS options'

        # 3. TLS handshake succeeds (self-signed)
        echo | openssl s_client -connect nextcloud.local:443 -
servername nextcloud.local 2>/dev/null | grep -i 'verify return
code'

        # 4. site reachable
        curl -sk -o /dev/null -w '%{http_code}\n'
https://nextcloud.local/login   # expect 200
```

Automation

The systemd user service applies the patch at every session start (idempotent):

```
        systemctl --user status nextcloud-aio-podman-fix.service
```

Caveats

- The `containers.json` patch and the Caddyfile bind mount are **lost when the mastercontainer is recreated** (`podman rm -f` + `podman run` , e.g. when updating). Always re-run `~/.local/bin/nextcloud-aio-podman-fix.sh` afterwards.
- Recreating the apache container changes its bridge IP → restart `nextcloud-aio-nextcloud` if you get HTTP `503` (see Fix 3).
- The certificate is self-signed: every client must accept the browser warning. For a trusted certificate on the LAN, put a local reverse proxy in front instead (see reverse-proxy.md).
- If you later switch to a public domain, remove the Caddyfile bind mount (and the `tls internal` template) again so real Let’s Encrypt certificates can be issued.

Yes, governments and public sector organizations have strong, structural incentives to prefer **Podman** over **Docker**. While both follow Open Container Initiative

(OCI) standards, Podman's architecture directly solves key compliance, security, and procurement challenges that public sector agencies face.

---

1. Principle of Least Privilege & Security Compliance

Government security standards (such as **FedRAMP**, **NIST SP 800-53**, **DISA STIGs**, and **SOC 2**) strictly enforce the principle of least privilege.

- **Rootless by Default:** Docker historically relies on a daemon running as `root`. Giving developers or services access to the Docker socket effectively grants them root-level system access. Podman was built to be **rootless by default** using Linux user namespaces. Containers run under normal, unprivileged user accounts.
- **No Central Daemon (Reduced Attack Surface):** Docker uses a centralized daemon process (`dockerd`) to manage containers. If the Docker daemon is compromised or crashes, every container on the host is at risk. Podman is **daemonless**—each container runs as a standard, isolated child process (fork/exec model), significantly shrinking the blast radius of any exploit.

---

2. Licensing, Procurement, and Zero Licensing Risk

Government IT procurement is subject to strict budget cycles and audit risks:

- **Open Source vs. Commercial Subscriptions:** Docker Desktop changed its business model, requiring paid subscriptions for enterprise organizations (over 250 employees or $10M revenue). Tracking hundreds or thousands of Docker Desktop seats across defense, civilian, and contractor networks creates complex audit and licensing overhead.
- **Podman is 100% Free and Open Source:** Developed under the Apache 2.0 license, Podman and Podman Desktop carry zero proprietary licensing friction or tier-based fees, making public sector software asset management trivial.

---

3. Native Integration with Government Linux Standards

In defense and government infrastructure, enterprise-grade, supported Linux distributions dominate:

- **Red Hat Enterprise Linux (RHEL) Alignment:** RHEL is standard across many Western governments and intelligence agencies due to its long-term support and compliance certifications (e.g., FIPS compliance). Since RHEL 8, **Podman is the official default container engine** and Docker is no longer natively shipped or supported by Red Hat.
- **Native `systemd` Integration:** Government operational playbooks rely heavily on `systemd` for managing system services, logs, and uptime. Podman generates and integrates natively with `systemd` unit files (`podman generate systemd`), allowing containers to be managed like standard system services.

---

4. Enhanced Auditability & Forensics

In high-security government operations, tracking *who* executed an action is critical for post-incident analysis:

- With Docker's client-server model, commands go through the daemon socket. In system logs, every action often traces back to the `dockerd` process running as root, obscuring which Linux user actually executed the command.
- Because Podman containers run directly as child processes of the invoking user, standard Linux auditing subsystems (`auditd`) can easily track individual user IDs down to exact container execution events.

Summary Comparison

Aspect	Docker	Podman	Government Impact
Architecture	Centralized Daemon (`dockerd`)	Daemonless (Fork/Exec)	Podman eliminates single point of failure

Aspect	Docker	Podman	Government Impact
Default Privilege	Root privileges usually required	Rootless by default	Podman satisfies least-privilege audits
Licensing	Paid subscription for larger teams	Free & Open Source (Apache 2.0)	Zero procurement overhead with Podman
Default in RHEL	No (Deprecated by Red Hat)	Yes (Default engine)	Seamless fit for enterprise government OS
Audit Log Trail	Obscured by central daemon	Direct user ID mapping (`auditd`)	Simplifies forensic investigations

---

Migration Friction

Because Podman was intentionally designed as an **OCI-compliant drop-in replacement**, government agencies don’t have to rewrite their container images or CLI tooling. Developers can literally alias `alias docker=podman` in their terminal and continue using standard `Containerfile` / `Dockerfile` formats without disrupting workflows.

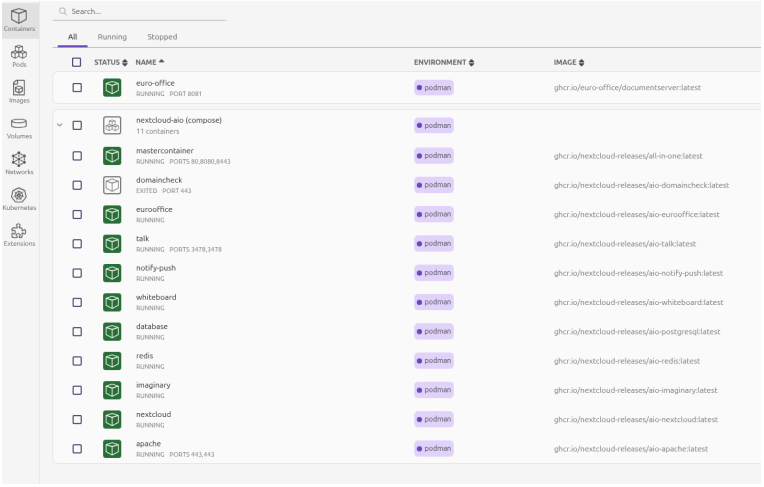
Nextcloud EuroOffice integration (document saving fix)

This document records how EuroOffice is integrated into this Nextcloud All-in-One (Podman, local-only) setup and which fix was needed so that documents can be **saved**. It is a companion to installation-steps.md and local-nextcloud-fixes.md.

Environment

- Linux Mint 22.3, user `naj` (uid/gid 1000), rootless Podman 4.9.3
- AIO mastercontainer with `SKIP_DOMAIN_VALIDATION=true`, local-only domain `nextcloud.local` → `192.168.0.114`
- TLS on the apache container uses Caddy's **internal CA** (self-signed) because no public certificate can be issued for a LAN-only domain (see local-nextcloud-fixes.md)
- EuroOffice runs as the AIO-managed container `nextcloud-aio-eurooffice` on the `nextcloud-aio` network (IP `10.89.3.13`, port 80), reachable from the browser via `https://nextcloud.local/eurooffice`
- EuroOffice app in Nextcloud: `eurooffice` 11.0.1, `DocumentServerUrl = https://nextcloud.local/eurooffice`, shared JWT secret injected by AIO

> There is an unused, standalone container `euro-office` (`ghcr.io/euro-office/documentserver`) publishing host port `8081`. It is **not** used by the EuroOffice app (which points to `https://nextcloud.local/eurooffice`). It can be removed to avoid confusion; it was left in place.

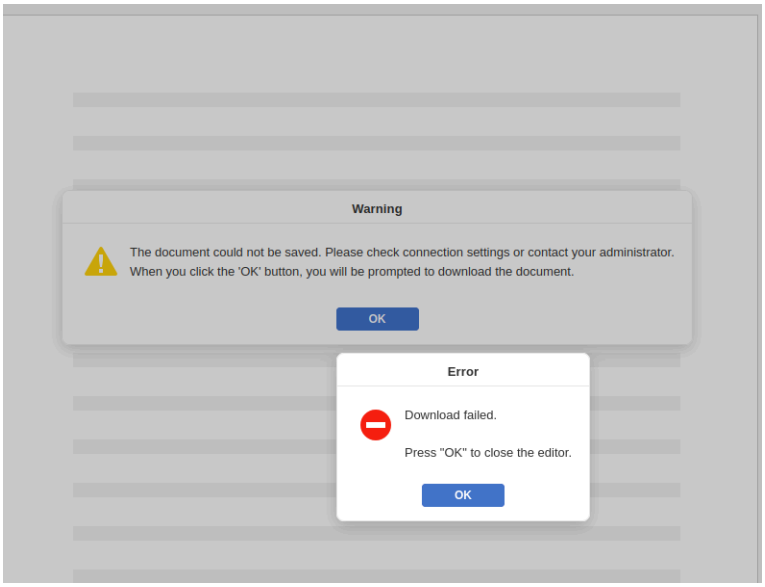


Running container: Euro-Office

Problem

Opening documents works (the browser can reach the documentserver), but **saving fails** with:

> The document cannot be saved. Please check connection settings or contact your administrator.



Error saving document as admin in Nextcloud

This is a callback/download failure: the documentserver's background services (converter, docservice) cannot talk to Nextcloud over the network. In this local-only setup the server-to-server requests use `https://nextcloud.local`, which:

1. is served with a **self-signed certificate** (Caddy internal CA), and
2. resolves to the **private IP** `192.168.0.114`.

Root cause (from the logs)

The converter log in the documentserver container shows the download failing at the TLS handshake:

```
error
downloadFile:url=https://nextcloud.local/apps/eurooffice/download?.
Error: unable to get local issuer certificate
```

The same TLS failure breaks the callback POST of the saved document back to Nextcloud. Relevant paths (in `nextcloud-aio-eurooffice`):

- Config: `/etc/euro-office/documentserver/local.json` (and `default.json`)
- Converter log: `/var/log/euro-office/documentserver/converter/out.log`
- Docservice log: `/var/log/euro-office/documentserver/docservice/out.log`

Mapping to the generic 4-step tutorial

Generic step	Applied?	How
1. `allow_local_remote_servers => true` in Nextcloud `config.php`	Already done	Already present in the generated `config.php`
2. `allowPrivateIPAddress: true` in the documentserver	Yes	via `ALLOW_PRIVATE_IP_ADDRE` (see below)
3. Internal routing URLs in the app	Not applicable	EuroOffice app 11.0.1 (shipped by AIO) has no setting
4. `rejectUnauthorized: false` / `SSL_INSECURE=true`	Yes	via `USE_UNAUTHORIZED_STORA` (see below)

Fix

The documentserver image's entrypoint (`/entrypoint.sh`) natively supports two environment variables that map to the required JSON keys and are applied to `local.json` **on every container start**:

Env var	JSON effect
`USE_UNAUTHORIZED_STORAGE=true`	`.services.CoAuthoring.requestDe = false`

Env var	JSON effect
`ALLOW_PRIVATE_IP_ADDRESS=true`	`.services.CoAuthoring["request-agent"].allowPrivateIPAddress =`

Two things were done: a **durable** change in `containers.json` (survives container recreates) and an **immediate** change in the running container (no recreate needed).

1. Durable: add the env vars to `nextcloud-aio-eurooffice` in `containers.json`

The container definitions live in `containers.json` inside the running mastercontainer (`/var/www/docker-aio/php/containers.json`). Add the two vars to the `environment` of `nextcloud-aio-eurooffice`:

```
        podman exec -i nextcloud-aio-mastercontainer python3 -
<<'PYEOF'
        import json
        path = '/var/www/docker-aio/php/containers.json'
        data = json.load(open(path))
        changed = False
        for c in data['aio_services_v1']:
            if c.get('container_name') == 'nextcloud-aio-
eurooffice':
                env = c.get('environment', [])
                for var in ('USE_UNAUTHORIZED_STORAGE=true',
'ALLOW_PRIVATE_IP_ADDRESS=true'):
                    if var not in env:
                        env.append(var)
                        changed = True
        json.dump(data, open(path, 'w'), indent=2)
        print('changed:', changed)
        PYEOF

        podman restart nextcloud-aio-mastercontainer   # clears
the containers.json cache
```

2. Immediate: edit `local.json` in the running container

This makes the fix effective without recreating the container:

```
        podman exec -i nextcloud-aio-eurooffice python3 -
<<'PYEOF'
        import json
        path = '/etc/euro-office/documentserver/local.json'
        data = json.load(open(path))
        co = data.setdefault('services',
{}).setdefault('CoAuthoring', {})
        co.setdefault('requestDefaults', {})
['rejectUnauthorized'] = False
        co.setdefault('request-filtering-agent', {})
['allowPrivateIPAddress'] = True
        json.dump(data, open(path, 'w'), indent=2)
        PYEOF

        podman restart nextcloud-aio-eurooffice
```

The manual edits survive a `podman restart`: the entrypoint's jq filter starts with `.` (identity) and only overrides the env-controlled keys, so the extra keys are preserved. They do **not** survive a container recreate - but then the env vars from step 1 regenerate `local.json` with the same values, so no manual step is needed.

3. Automation: keep the fix across mastercontainer recreates

The `containers.json` patch is **lost when the mastercontainer is recreated** (`podman rm -f` + `podman run`, e.g. on update). The helper script `~/.local/bin/nextcloud-aio-podman-fix.sh` (which already patches `cap_add` and the Caddyfile bind mount) was extended to also (re-)apply the two EuroOffice env vars. Run it after every mastercontainer recreate:

```
        ~/.local/bin/nextcloud-aio-podman-fix.sh
```

It is idempotent (`containers.json already patched` when nothing changes).

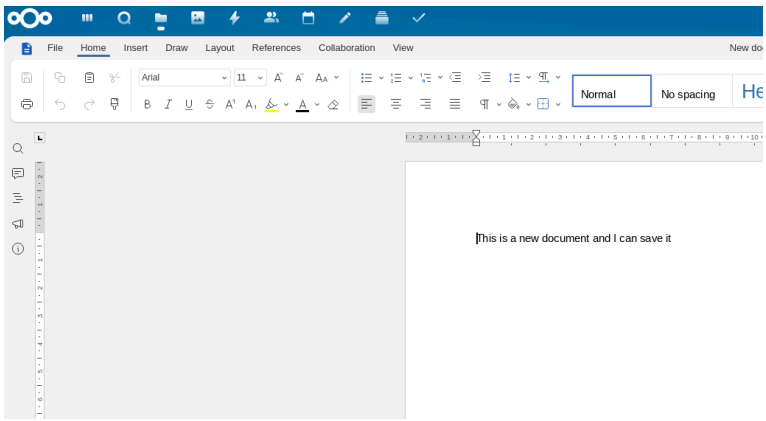
Verification

```
        # 1. env vars are in the container definition
        podman exec nextcloud-aio-mastercontainer python3 -c "
        import json
        d=json.load(open('/var/www/docker-
aio/php/containers.json'))
        for c in d['aio_services_v1']:
            if c['container_name']=='nextcloud-aio-eurooffice':
                env=' '.join(c.get('environment',[]))
                print('USE_UNAUTHORIZED_STORAGE=true:',
'USE_UNAUTHORIZED_STORAGE=true' in env)
                print('ALLOW_PRIVATE_IP_ADDRESS=true:',
'ALLOW_PRIVATE_IP_ADDRESS=true' in env)
        "

        # 2. effective JSON in the running container
        podman exec nextcloud-aio-eurooffice bash -lc "python3 -
c \"
        import json
        d=json.load(open('/etc/euro-
office/documentserver/local.json'))
        co=d['services']['CoAuthoring']
        print('rejectUnauthorized:', co['requestDefaults']
['rejectUnauthorized'])
        print('allowPrivateIPAddress:', co['request-filtering-
agent']['allowPrivateIPAddress'])
        \""

        # 3. containers healthy
        podman inspect nextcloud-aio-eurooffice --format
'{{.State.Status}} {{.State.Health.Status}}'
        podman inspect nextcloud-aio-mastercontainer --format
'{{.State.Status}} {{.State.Health.Status}}'
```

Final check: open a document in EuroOffice and save it.



Saving documents in Nextcloud works

Caveats

- The `containers.json` patch is lost when the **mastercontainer is recreated** - re-run `~/.local/bin/nextcloud-aio-podman-fix.sh` afterwards (the systemd user service `nextcloud-aio-podman-fix.service` already does this at each session start).

- Disabling `rejectUnauthorized` is acceptable only for a **local-only** instance with a self-signed certificate. If this setup ever moves to a public domain with a real Let's Encrypt certificate, the vars can be removed again (and local.json edits reverted) so TLS verification stays enabled.
- The unrelated Talk **standalone-signaling** server-side requests to `https://nextcloud.local` show the same `CERTIFICATE_VERIFY_FAILED` errors in `nextcloud.log`; that is a separate issue, not covered here.

Yes, Euro-Office can be integrated into Nextcloud and Nextcloud All-in-One (AIO). Nextcloud added Euro-Office as an alternative office suite alongside options like Collabora Online and Nextcloud/ONLYOFFICE.

Integration Options

Option 1: Fresh Installation via Nextcloud AIO

In the latest versions of the Nextcloud All-in-One setup interface (the setup wizard running on port `8080`), Euro-Office is selectable directly as a bundled container option during initial setup:

1. Launch the Nextcloud AIO interface before finalizing the initial container deployment.
2. Under the **Optional Add-ons / Office Suite** section, toggle on **Euro-Office**.
3. Click **Start Containers**. AIO will automatically spin up the Euro-Office document server container and handle internal network routing and SSL certificates for you.

Option 2: Existing Nextcloud AIO Setup (App Store)

If your AIO instance is already running:

1. **Enable the Integration App in Nextcloud:**

- Log in as an administrator.
- Go to **Top-Right Profile Menu → Apps → Office & text**.
- Find **Euro-Office integration** (or search for `eurooffice`) and click **Download and enable**.

2. **Connect to the Euro-Office Document Server:**

- Go to **Administration settings → Euro-Office**.
- Enter the address of your Euro-Office Document Server instance (if self-hosting externally) along with your Secret Key / JWT token.
- Save the configuration.

Euro-Office Installation Guide

This document provides detailed instructions for deploying Euro-Office, a self-hostable office suite consisting of a document server and Nextcloud integration.

Table of Contents

Overview

Euro-Office consists of two main components:

1. **Document Server** - Real-time collaboration server for office documents
2. **Nextcloud App** - Integration layer for Nextcloud

System Requirements

Document Server

Requirement	Specification
RAM	4 GB minimum
Disk Space	10 GB minimum
Supported Platforms	Ubuntu 24.04 LTS, Debian 12, Fedora 41+, Docker

Nextcloud Server

- Nextcloud 25 or later
- PHP 8.0+
- PostgreSQL or MySQL database

Installation Methods

Docker Installation

Prerequisites: - Docker Engine 20.10 or later - 5 GB disk space for the image - 4 GB RAM minimum

Quick Start:

```
docker run -d \
  --name euro-office \
  --restart=unless-stopped \
  -p 80:80 \
  -e JWT_ENABLED=true \
  -e JWT_SECRET=at-least-32-chars-long-for-hs256 \
  ghcr.io/euro-office/documentserver:latest
```

Verify Installation:

```
curl http://localhost/healthcheck
```

Persistent Data:

```
docker run -d \
  --name euro-office \
  --restart=unless-stopped \
  -p 80:80 \
  -e JWT_ENABLED=true \
  -e JWT_SECRET=at-least-32-chars-long-for-hs256 \
  -v /path/to/data:/var/lib/euro-office/documentserver \
  -v /path/to/logs:/var/log/euro-office/documentserver \
  -v /path/to/config:/etc/euro-office/documentserver \
  ghcr.io/euro-office/documentserver:latest
```

Environment Variables:

Variable	Default	Description
`JWT_ENABLED`	`true`	Enable JWT authentication
`JWT_SECRET`	—	Shared secret for JWT
`JWT_HEADER`	`Authorization`	HTTP header carrying the JWT
`EXAMPLE_ENABLED`	`false`	Enable built-in example app
`WOPI_ENABLED`	`false`	Enable WOPI protocol support

Ubuntu Installation

Step 1 — Install Prerequisites:

```
sudo apt-get update
sudo apt-get install -y postgresql redis-server
rabbitmq-server nginx supervisor
```

Step 2 — Create Database:

```
        sudo -u postgres psql -c "CREATE USER ds WITH PASSWORD
'ds';"
        sudo -u postgres psql -c "CREATE DATABASE ds OWNER ds;"
```

Step 3 — Pre-seed Installer Answers:

```
        echo "ds ds/db-type select postgres
        ds ds/db-host string localhost
        ds ds/db-port string 5432
        ds ds/db-user string ds
        ds ds/db-pwd password ds
        ds ds/db-name string ds" | sudo debconf-set-selections
```

Step 4 — Download Package:

Replace `<version>` and `<arch>` with your values (e.g., `9.3.1-rc.1` and `amd64` or `arm64`):

```
        wget "https://github.com/Euro-
Office/DocumentServer/releases/download/v<version>/euro-office-
documentserver_<version>_<arch>.deb" \
          -O /tmp/euro-office-documentserver.deb
```

Step 5 — Install Package:

```
        sudo apt-get install -y /tmp/euro-office-
documentserver.deb
```

Step 6 — Verify:

```
        systemctl is-active ds-docservice ds-converter ds-
metrics nginx
        curl http://localhost/healthcheck
```

Fedora Installation

Step 1 — Install Prerequisites:

```
        sudo dnf install -y \
          postgresql-server postgresql postgresql-contrib \
          valkey \
          rabbitmq-server \
          nginx \
          supervisor

        sudo postgresql-setup --initdb
        sudo systemctl enable --now postgresql valkey rabbitmq-
server nginx supervisor
```

Step 2 — Configure PostgreSQL Authentication:

Edit `/var/lib/pgsql/data/pg_hba.conf` and change `ident` to `md5` on the `127.0.0.1` and `::1` lines.

Step 3 — Create Database:

```
        sudo -u postgres psql -c "CREATE USER ds WITH PASSWORD
'ds';"
        sudo -u postgres psql -c "CREATE DATABASE ds OWNER ds;"
```

Step 4 — Download Package:

```
        wget "https://github.com/Euro-
Office/DocumentServer/releases/download/v<version>/euro-office-
documentserver-<version>.x86_64.rpm" \
          -O /tmp/euro-office-documentserver.rpm
```

Step 5 — Install Package:

```
        sudo rpm -ivh --nodeps /tmp/euro-office-
documentserver.rpm
```

Step 6 — Initialize Database Schema:

```
        sudo -u postgres psql -d ds \
          -f
/var/www/onlyoffice/documentserver/server/schema/postgresql/createdb.sql

        sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON ALL
TABLES IN SCHEMA public TO ds;"
        sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON ALL
SEQUENCES IN SCHEMA public TO ds;"
```

Step 7 — Fix nginx Configuration:

Edit `/etc/nginx/nginx.conf` and: 1. Remove or comment out `include /etc/nginx/conf.d/*.conf;` in the `http {}` block 2. Comment out the default `server {}` block listening on port 80

Step 8 — Install OpenSSL and Generate Caches:

```
        sudo dnf install -y openssl
        sudo /usr/bin/documentserver-flush-cache.sh
```

Step 9 — Configure JWT Authentication:

Create `/etc/euro-office/documentserver/local.json` with your JWT secret.

Step 10 — Start Services:

```
        sudo systemctl enable --now ds-docservice ds-converter
ds-metrics
```

Nextcloud App Installation

Step 1 — Navigate to Nextcloud Apps Directory:

```
cd /path/to/nextcloud/apps/
```

Step 2 — Clone Repository:

```
git clone https://github.com/Euro-Office/eurooffice-nextcloud.git eurooffice
cd eurooffice
git submodule update --init --recursive
```

Step 3 — Build Frontend Assets:

```
npm install
npm run build
```

Step 4 — Install Composer Dependencies:

```
composer install
```

Step 5 — Set Permissions:

```
chown -R www-data:www-data eurooffice
```

Step 6 — Enable in Nextcloud:

1. Open Nextcloud admin panel
2. Navigate to Settings → Apps → Disabled apps
3. Find **Euro-Office** and click **Enable**

Configuration

Via Nextcloud Admin UI

1. Open Settings → Admin → Euro-Office
2. Enter Document Server URL:
   `https://<documentserver>/`
3. Configure JWT secret (must match Document Server)

Via occ Command

```
php occ config:app:set eurooffice DocumentServerUrl --value="https://<documentserver>/"
php occ config:app:set eurooffice jwt_secret --value="your_secret_key"
```

Via config.php

Add to `config/config.php`:

```
        "eurooffice" => array (
            "DocumentServerUrl" => "https://<documentserver>/",
            "jwt_secret" => "your_secret_key",
        )
```

Key Settings

Setting	Description	Example
`DocumentServerUrl`	Document Server address	`https://doc.example.com/`
`jwt_secret`	JWT secret key	`your_secret_key`
`sameTab`	Open in same tab	`false`
`enableSharing`	Enable file sharing	`true`
`preview`	Use for previews	`true`

Verification

Check Connection:

```
        occ eurooffice:documentserver --check
```

Test with Example App (Docker):

```
        docker run -d \
          --name euro-office-test \
          -p 8080:80 \
          -e JWT_ENABLED=true \
          -e JWT_SECRET=test_secret \
          -e EXAMPLE_ENABLED=true \
          ghcr.io/euro-office/documentserver:latest
```

Open `http://localhost:8080/example/` in browser.

Troubleshooting

Self-Signed Certificates

If using self-signed certificates, either:

1. Check **Disable certificate verification** in Nextcloud admin panel, or
2. Add to `config/config.php`:

```
        "eurooffice" => array (
            "verify_peer_off" => true
        )
```

Editors Not Opening

If editors stop working after initial success:

1. Check network settings and SSL certificates
2. Verify Document Server is running:

```
        systemctl is-active ds-docservice ds-converter ds-
metrics nginx
```

Connection Check Interval

Modify the background check interval in `config/config.php`:

```
        "eurooffice" => array (
            "editors_check_interval" => 3624  // in seconds
        )
```

Set to `0` to disable.

Additional Resources

- Euro-Office Documentation
- Nextcloud App Repository
- Document Server Repository

Dat is een hele bekende drempel bij de installatie van **Nextcloud All-in-One (AIO)**!

Nextcloud AIO verwacht in de standaard (“normal mode”) een **volledige, publieke domeinnaam** (zoals `nextcloud.jouwdomein.nl`), omdat de ingebouwde proxy automatisch geldige SSL-certificaten (HTTPS) probeert aan te vragen via *Let’s Encrypt*. Een lokaal IP-adres (zoals `192.168.x.x`) of `localhost` accepteert dit veld in deze stand **niet**.

🛠️ Welke optie past bij jouw situatie?

Afhankelijk van wat je precies wilt bereiken op je lokale netwerk, zijn er drie manieren om dit op te lossen:

Optie 1: Je wilt Nextcloud alléén lokaal gebruiken (zonder internettoegang)

Als je de server niet wilt blootstellen aan het internet en enkel binnen je eigen wifi/netwerk wilt gebruiken, moet je AIO vertellen dat de SSL-check overgeslagen mag worden.

- **Oplossing:** Je moet het `docker run` commando waarmee je AIO hebt opgestart uitbreiden met een extra omgevingsvariabele: `-e SKIP_DOMAIN_VALIDATION=true`
- **Wat vul je daarna in het veld in?** Je vult dan de lokale hostname of het lokale IP-adres van je pc in (bijv. `192.168.1.100` of `mijnpc.local`).

> 💡 **Let op:** Je browser geeft dan wel een certificaatwaarschuwing (“Verbinding niet veilig”), omdat je een zelfondertekend SSL-certificaat gebruikt.

Optie 2: Je hebt een eigen domeinnaam (bijv. bij TransIP, MijnDomein, Cloudflare)

Als je je Nextcloud ook vanaf buitenaf wilt kunnen bereiken:

1. Vul in het veld jouw (sub)domein in, bijv: `nextcloud.jouwdomein.nl`.
2. Zorg dat je in het beheerderspaneel van je domein een **A-record** aanmaakt die wijst naar het **externe IP-adres van jouw thuisnetwerk** (je publieke IP).
3. Stuur op je internetrouter **poort 80 en poort 443** (TCP) door (port forwarding) naar het lokale IP-adres van je computer.

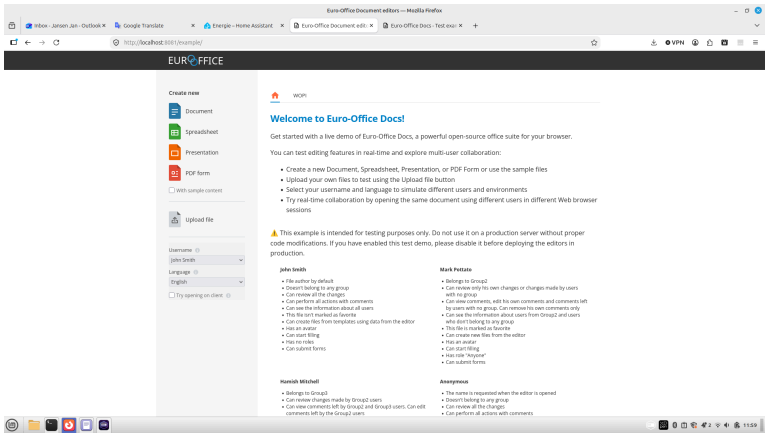
Optie 3: Je wilt een gratis publiek domein (deSEC of DuckDNS)

Als je geen eigen domein hebt, maar wél van buitenaf toegang wilt via HTTPS:

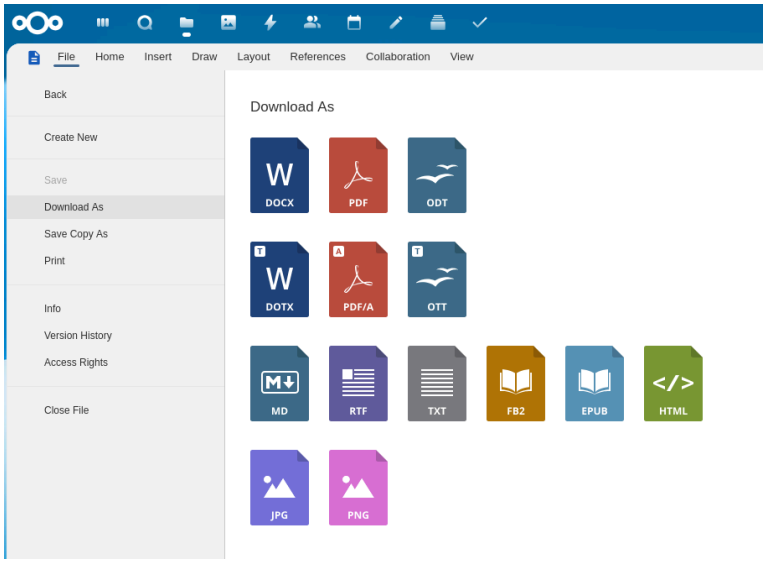
1. Maak een gratis domein aan via deSEC.io (deze link staat ook in je Nextcloud scherm) of DuckDNS.org. Je krijgt dan bijvoorbeeld `mijnnextcloud.dedyn.io`.
2. Koppel jouw publieke IP-adres aan dit gratis domein.
3. Vul `mijnnextcloud.dedyn.io` in het veld in.
4. Vergeet niet om **poort 80 en poort 443** in je router open te zetten/door te sturen naar deze computer.

Appendix: Screenshots of the integrated Euro-Office suite

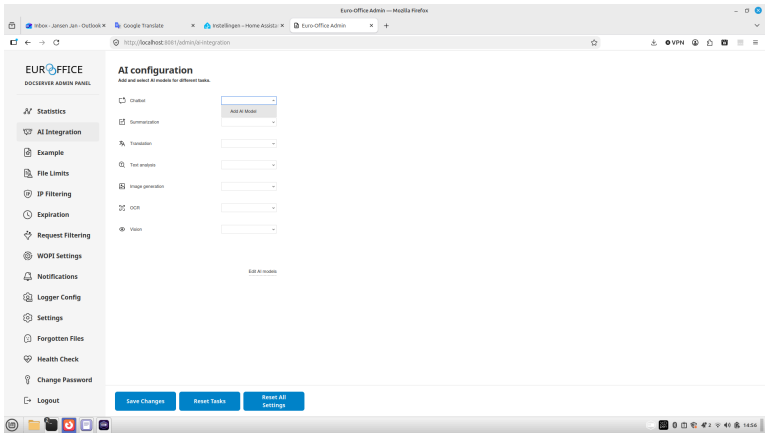
The following screenshots, taken from the running installation, show the Euro-Office office suite integrated into Nextcloud.



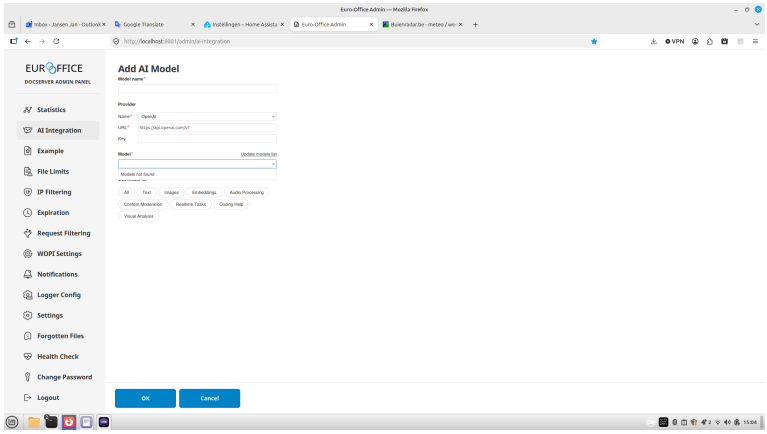
Example: Euro-Office



Document formats



Euro-Office AI configuration



Add AI model